

Отчёт по лабораторной работе №4

Реализация основных моделей в агентном подходе

true

Цель работы

Освоить реализацию эпидемиологической модели SIR в агентном подходе на языке Julia с использованием пакета Agents.jl[@datseris2022agents]. Научиться проводить параметрическое сканирование и многокритериальную оптимизацию агентных моделей, а также оформлять скрипты в литературном стиле с помощью Literate.jl и Quarto.

Задание

- Создать рабочий каталог и инициализировать проект DrWatson.
- Установить пакеты: DrWatson, Agents, DataFrames, Plots, CSV, JLD2, BlackBoxOptim, Distributions.
- Запустить базовый эксперимент агентной модели SIR на 100 шагов.
- Провести параметрическое сканирование коэффициента заразности β от 0.1 до 1.0.
- Исследовать влияние интенсивности миграции между городами на пик заболеваемости.
- Выполнить многокритериальную оптимизацию параметров с помощью BlackBoxOptim.
- Преобразовать все скрипты в литературный стиль, сгенерировать Jupyter Notebook и Quarto-документы.
- Интегрировать результаты в отчёт.

Теоретическое введение

Агентное моделирование эпидемий

В агентном подходе каждый индивид моделируется как отдельный агент со своим состоянием. В отличие от детерминированных моделей на дифференциальных уравнениях, здесь взаимодействия стохастичны, а агенты могут мигрировать между узлами графа. В данной работе агенты размещены на графе из трёх городов, соединённых рёбрами с заданной матрицей переходов.

Модель SIR

Модель делит популяцию на три компартмента:

- **S** — восприимчивые,
- **I** — инфицированные,
- **R** — выздоровевшие.

Классические уравнения детерминированной модели [@kermack1927contribution]:

$$\frac{dS}{dt} = -\beta \frac{SI}{N}, \quad \frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I$$

где β — коэффициент заразности, γ — скорость выздоровления. Пороговый параметр $R_0 = \beta/\gamma$: при $R_0 > 1$ эпидемия развивается, при $R_0 < 1$ затухает.

В агентной реализации каждый агент типа `Person` имеет два поля: `days_infected::Int` и `status::Symbol (:S, :I или :R)`. Заражение происходит с разными коэффициентами в зависимости от того, выявлен ли носитель: β_{und} для невыявленных и β_{det} для выявленных.

Используемые пакеты

Table 1: Пакеты Julia [bezanson2017julia], используемые в лабораторной работе {tbl-packages}

Пакет	Назначение
DrWatson	Организация проекта и воспроизводимость экспериментов
Agents.jl	Агентное моделирование на графах
DataFrames	Хранение и обработка табличных данных
Plots	Визуализация

Пакет	Назначение
CSV	Чтение и запись данных
JLD2	Сохранение объектов Julia
BlackBoxOptim	Оптимизация методом чёрного ящика
Distributions	Вероятностные распределения
Literate.jl	Литературное программирование

Выполнение лабораторной работы

Инициализация проекта и установка пакетов

— Перешёл в каталог `labs/lab04` и запустил Julia 1.12.6. Выполнил `using DrWatson` и `initialize_project("project")` — в результате создана структура каталогов проекта и автоматически установлен DrWatson v2.19.1 вместе с зависимостями: JLD2 v0.4.6, FileIO v1.19.0, MacroTools v0.5.16, ChunkCodecCore v1.0.1 и другими ([рис. @fig-001]). При обновлении реестра возникли предупреждения о недоступности некоторых серверов, однако это не помешало установке.

Запуск Julia 1.12.6, инициализация проекта DrWatson, установка зависимостей

Figure 1: Запуск Julia 1.12.6, инициализация проекта DrWatson, установка зависимостей

— Дополнительно установил пакет Distributions v0.25.126 командой `Pkg.add("Distributions")`. Предкомпиляция заняла 30 секунд при 546 уже скомпилированных пакетах. Затем выполнил `include("src/sir_model.jl")` для проверки корректности модели — в ответ получил `total_count (generic function with 1 method)`, что подтверждает успешную загрузку ([рис. @fig-002]).

Установка Distributions v0.25.126 и проверочный запуск `sir_model.jl`

Figure 2: Установка Distributions v0.25.126 и проверочный запуск `sir_model.jl`

Реализация модели SIR

— В VS Code открыл файл `src/sir_model.jl` ([рис. @fig-003]). Агент описывается макросом `@agent struct Person(GraphAgent)` с двумя полями: `days_infected::Int` и `status::Symbol`. Функция

`initialize_sir` принимает параметры по умолчанию: три города по 1000 агентов ($N_s = [1000, 1000, 1000]$), $\beta_{und} = [0.5, 0.5, 0.5]$, $\beta_{det} = [0.05, 0.05, 0.05]$, период инфекции 14 дней, время выявления 7 дней, смертность 0.02, вероятность повторного заражения 0.1. Инфекция стартует в третьем городе: $I_s = [0, 0, 1]$.

Файл `sir_model.jl`: структура агента `Person` и параметры `initialize_sir`

Figure 3: Файл `sir_model.jl`: структура агента `Person` и параметры `initialize_sir`

Базовый эксперимент

— Скрипт `sir_run_basic.jl` задаёт словарь `params` с теми же параметрами по умолчанию и запускает симуляцию на 100 шагов ([рис. @fig-004]). На каждом шаге собираются данные о численности S , I , R . По завершении результаты сохранены в `data/sir_basic_agent.jld2` и `data/sir_basic_model.jld2`, которые видны в файловом менеджере ([рис. @fig-005]).

Скрипт `sir_run_basic.jl`: параметры модели и инициализация

Figure 4: Скрипт `sir_run_basic.jl`: параметры модели и инициализация

Файловый менеджер: сохранённые файлы данных JLD2

Figure 5: Файловый менеджер: сохранённые файлы данных JLD2

— Скрипт запущен через `include("scripts/sir_run_basic.jl")` в Julia REPL ([рис. @fig-006]).

— Результирующий график динамики SIR показан на ([рис. @fig-025]). Видно, что в первые 15 дней кривая восприимчивых S (синяя) резко падает примерно с 3000 до нуля, а кривая инфицированных I (оранжевая) проходит через пик около 15-го дня и затем снижается. Кривая выздоровевших R (зелёная) нарастает волнами — это связано со стохастическим характером модели и механизмом повторного заражения. Линия «Всего» (пунктирная) постепенно снижается от 3000 к примерно 2600, что отражает накопленную смертность за 100 шагов.

Запуск `sir_run_basic.jl` в Julia REPL

Figure 6: Запуск `sir_run_basic.jl` в Julia REPL

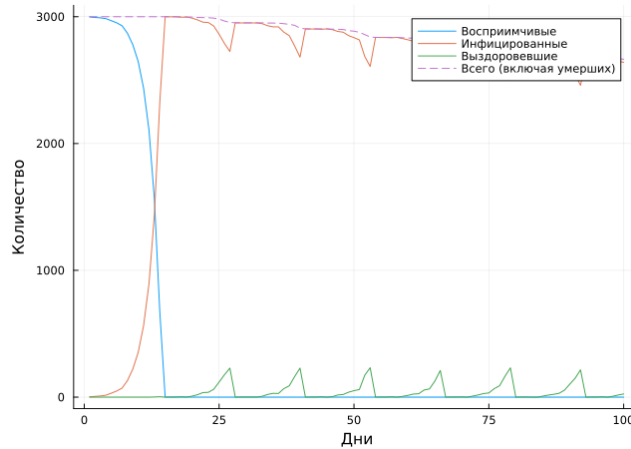


Figure 7: Базовая динамика SIR за 100 шагов: восприимчивые, инфицированные, выздоровевшие

Сканирование коэффициента заразности β

— В VS Code открыт скрипт `sir_scan_beta.jl` ([рис. @fig-007]). Функция `run_experiment(p)` принимает словарь параметров, формирует $\beta_{\text{und}} = \text{fill}(\beta, 3)$ и $\beta_{\text{det}} = \text{fill}(\beta/10, 3)$, инициализирует модель и считает пик заражённых за 100 шагов. Сканирование выполнялось по β от 0.1 до 1.0 с тремя репликами (seed 42, 43, 44).

Скрипт `sir_scan_beta.jl`: функция `run_experiment` и формирование векторов β

Figure 8: Скрипт `sir_scan_beta.jl`: функция `run_experiment` и формирование векторов β

— В терминале последовательно выводились сообщения о завершении 30 экспериментов ([рис. @fig-008]). Справа на том же скриншоте виден промежуточный график, на котором уже прослеживается пороговый характер зависимости. Результаты сохранены в `data/beta_scan_all.csv` и `plots/beta_scan.png`.

— По итоговому графику ([рис. @fig-026]) видно, что при $\beta = 0.1$ эпидемия не возникает — пик и конечная доля инфицированных практически равны нулю. При $\beta = 0.2$ пик резко вырастает

Терминальный вывод сканирования β и предварительный график

Figure 9: Терминальный вывод сканирования β и предварительный график

примерно до 0.33, а уже при $\beta \geq 0.5$ обе кривые (пик и конечная доля) стабилизируются вблизи 1.0, то есть через эпидемию проходит вся популяция. Доля умерших (зелёные ромбы) растёт медленнее и к $\beta = 1.0$ достигает примерно 0.14. Резкий скачок при $\beta \approx 0.2$ соответствует переходу через порог $R_0 = 1$.

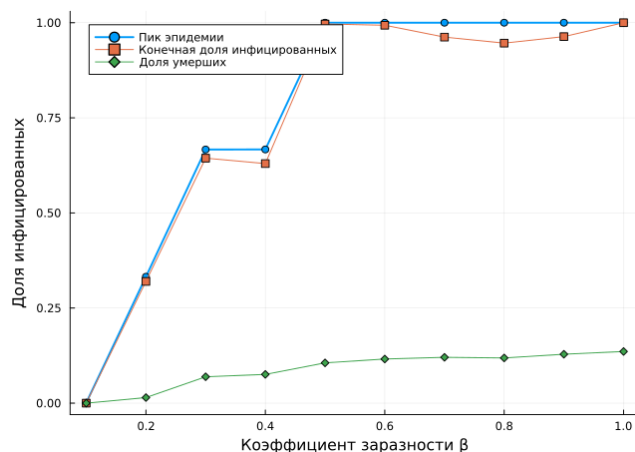


Figure 10: Сканирование по β : пик заболеваемости, конечная доля инфицированных и доля умерших

Исследование эффекта миграции

— В скрипте `sir_migration_effect.jl` реализована функция `create_migration_matrix(C, intensity)`, которая строит матрицу $C \times C$: внедиагональные элементы равны $\text{intensity}/(C - 1)$, диагональные — $1 - \text{intensity}$ (рис. @fig-009). Функция `peak_time(p)` запускает модель с заданной матрицей миграции и фиксирует время достижения пика.

Скрипт `sir_migration_effect.jl`: формирование матрицы миграции и функция эксперимента

Figure 11: Скрипт `sir_migration_effect.jl`: формирование матрицы миграции и функция эксперимента

— Эксперимент охватывал `migration_intensity` от 0.0 до 0.5 с шагом 0.1 и тремя репликами, итого 18 прогонов ([рис. @fig-010]). Результаты сохранены в `data/migration_scan_all.csv` и `plots/migration_effect.png`.

Терминальный вывод: 18 завершённых экспериментов по миграции

Figure 12: Терминальный вывод: 18 завершённых экспериментов по миграции

— На графике ([рис. @fig-027]) наблюдается резкий скачок пиковой заболеваемости уже при `intensity = 0.1`: она вырастает примерно с 1000 (без миграции) до ~3000, то есть в три раза. При значениях от 0.1 до 0.5 пик остаётся около 3000 — это означает, что при любом ненулевом потоке мигрантов инфекция быстро распространяется из третьего города во все остальные. Кривая «Время пика» (синяя) при этом остаётся вблизи нуля по всей оси, то есть пик фиксируется в самом начале симуляции при любой интенсивности.

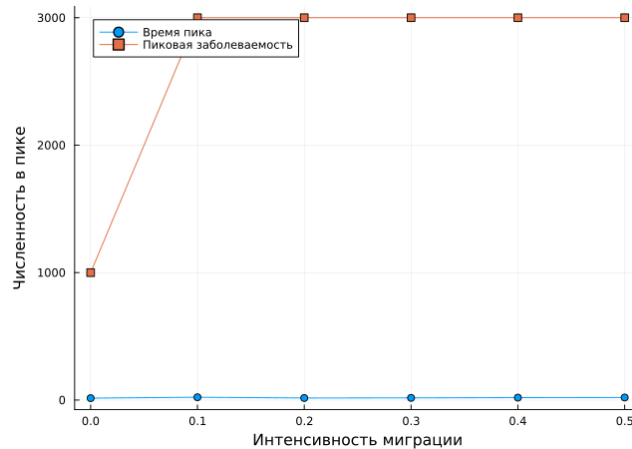


Figure 13: Эффект миграции: пиковая заболеваемость и время её достижения

Многокритериальная оптимизация

— Скрипт `sir_optimize_parameters.jl` определяет функцию `cost_multi(x)`, принимающую вектор из трёх параметров: β_{und} , времени выявления и смертности ([рис. @fig-011]). Для каждого набора выполняется 5 реплик, после чего вычисляются средний

пик заражённых и среднее число умерших (`dead_count = 3000 - nagents(model)`). `BlackBoxOptim` ищет Парето-фронт, минимизируя оба критерия одновременно.

Скрипт `sir_optimize_parameters.jl`: целевая функция `cost_multi`

Figure 14: Скрипт `sir_optimize_parameters.jl`: целевая функция `cost_multi`

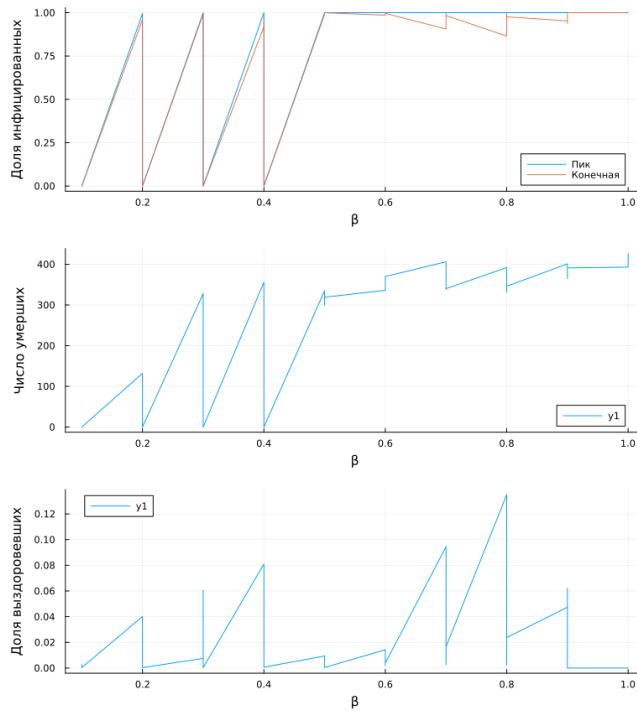
Сводная визуализация результатов

— Скрипт `sir_visualize_dynamics.jl` считывает `data/beta_scan_all.csv` и строит составной график из трёх субплотов: пик и конечная доля инфицированных; число умерших; доля выздоровевших ([рис. @fig-012]).

Скрипт `sir_visualize_dynamics.jl`: загрузка CSV и построение составного графика

Figure 15: Скрипт `sir_visualize_dynamics.jl`: загрузка CSV и построение составного графика

— Итоговый составной график ([рис. @fig-028]) отличается от `beta_scan.png` тем, что отражает полную вариабельность трёх реплик. На верхнем субплоте хорошо виден пороговый переход и разброс между репликами при $\beta = 0.3-0.4$. Средний субплот показывает, что число умерших при $\beta = 0.7-0.8$ достигает 400 человек из 3000. Нижний субплот отражает долю выздоровевших: пик около 0.13 при $\beta \approx 0.8$, после чего значение снижается — вероятно, из-за роста смертности при высоких β .



Полное

параметрическое сканирование

— Скрипт `sir_parametric_scan.jl` расширяет сканирование с `sir_scan_beta.jl`: вместо одного параметра β перебирается небольшой грид по β_{und} , `detection_time`, `death_rate` (при фиксированных `infection_period = 14` и `reinfection_probability = 0.1`). Грид сокращён до 12 комбинаций ($2 \times 3 \times 2$), чтобы расчёт укладывался в разумное время на слабой виртуальной машине.

— Результаты сохранены в `data/sir_parametric_scan_all.csv` с колонками β_{und} , `detection_time`, `death_rate`, `peak`, `final_inf`, `final_rec`, `deaths`, а также производными `gamma` и `R0`.

Сравнение с теоретическим порогом $R_0 = 1$

Для агентной модели $\gamma = 1/\text{infection_period} = 1/14 \approx 0.071$, следовательно теоретический порог по β составляет $\beta^* = \gamma \cdot R_0 = 0.071 \cdot 1 \approx 0.071$.

В параметрическом сканировании при $\beta_{\text{und}} = 0.2$ значение $R_0 = \beta/\gamma \approx 2.8$, что заметно выше теоретического порога — и пик эпидемии действительно ненулевой при всех значениях `detection_time`. При $\beta_{\text{und}} = 0.5$ ($R_0 \approx 7$) эпидемия охватывает практически всю популяцию независимо от `detection_time`.

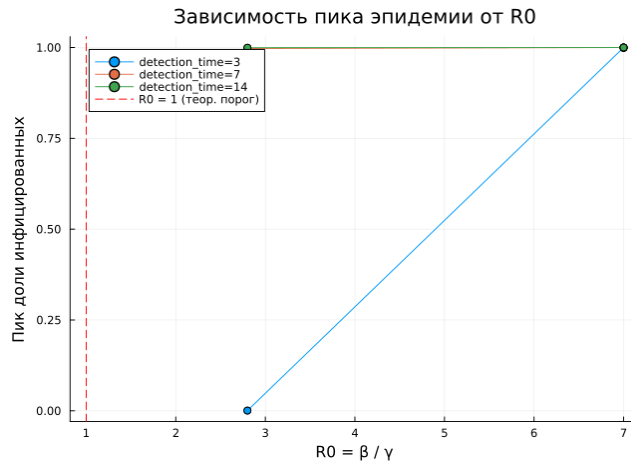


Figure 16: Зависимость пика заболеваемости от R_0 при разных `detection_time`

Ранее в сканировании `sir_scan_beta.jl` эмпирический порог возникновения эпидемии наблюдался между $\beta = 0.1$ и $\beta = 0.2$ (соответствует $R_0 \approx 1.4-2.8$), что несколько выше теоретического значения $R_0 = 1$ ($\beta^* \approx 0.071$). Это расхождение объясняется стохастической природой агентной модели: при малом числе изначально инфицированных ($I_s = [0, 0, 1]$, то есть один агент) и небольшой длительности симуляции цепочка заражений может оборваться случайным образом даже при R_0 , формально превышающем 1 — детерминированный порог $R_0 = 1$ справедлив для усреднённой по бесконечному числу повторов динамики, тогда как одиночный конечный прогон подвержен эффектам малых чисел.

```
{ {< include ../project/markdown/sir_parametric_scan/sir_parametric_scan.qmd
> } }
```

Преобразование скриптов в литературный стиль

— Все скрипты переработаны в литературный стиль с помощью `Literate.jl`. Ключевое отличие — комментарии вида `# #` задают заголовок модуля, `# ##` — подраздел. При обработке `Literate.jl` превращает их в markdown-заголовки и генерирует три артефакта: чистый `.jl`, ноутбук `.ipynb` и документ `.qmd`.

— Файл `sir_run_basic.jl` в литературном стиле ([рис. @fig-013]) разбит на блоки: «Базовый эксперимент модели SIR», «Подготовка окружения», «Задание параметров модели»,

«Инициализация модели», «Подготовка структур для сбора данных». Видно, что сам код не изменился — добавлены только структурированные комментарии.

`sir_run_basic.jl` в литературном стиле

Figure 17: `sir_run_basic.jl` в литературном стиле

— Файл `sir_migration_effect_literate.jl` ([рис. @fig-014]) содержит блоки «Формирование матрицы миграции» и «Определение функции эксперимента».

`sir_migration_effect_literate.jl` в литературном стиле

Figure 18: `sir_migration_effect_literate.jl` в литературном стиле

— Файл `sir_optimize_parameters_literate.jl` ([рис. @fig-015]) структурирован как «Многокритериальная оптимизация параметров» с подразделом «Определение целевой функции». Функция `cost_multi` занимает центральное место.

`sir_optimize_parameters_literate.jl` в литературном стиле

Figure 19: `sir_optimize_parameters_literate.jl` в литературном стиле

— Файл `sir_scan_beta_literate.jl` ([рис. @fig-016]) описывает сканирование коэффициента заразности и содержит функцию `run_experiment`.

— Файл `sir_visualize_dynamics_literate.jl` ([рис. @fig-017]) разбит на блоки «Загрузка результатов» и «Построение графиков».

Jupyter Notebook

— После генерации в JupyterLab (порты 8888–8891) открыты ноутбуки для всех скриптов.

— Ноутбук `sir_migration_effect_literate.ipynb` ([рис. @fig-018]) воспроизводится на Julia 1.12. В первой ячейке загружены зависимости и вызван `include(scrdir("sir_model.jl"))`, ответ — `total_count (generic function with 1 method)`.

— Ноутбук `sir_run_basic.ipynb` ([рис. @fig-019]) содержит словарь параметров, который совпадает с тем, что задан в `sir_run_basic.jl`: `:Ns => [1000, 1000, 1000]`, `:β_und => [0.5, 0.5, 0.5]`, `:n_steps => 100`.

sir_scan_beta_literate.jl в литературном стиле

Figure 20: sir_scan_beta_literate.jl в литературном стиле

sir_visualize_dynamics_literate.jl в литературном стиле

Figure 21: sir_visualize_dynamics_literate.jl в литературном стиле

— Ноутбук `sir_scan_beta_literate.ipynb` ([рис. @fig-020]) воспроизводит функцию `run_experiment` и подключает те же зависимости, что и исходный скрипт.

— В ноутбуке `sir_visualize_dynamics_literate.ipynb` ([рис. @fig-021]) результат загрузки `beta_scan_all.csv` виден как `DataFrame` размером 30×13 . Среди столбцов — `beta`, `peak`, `deaths`, `final_rec`, `final_inf`, `seed`, `death_rate`, `infection_period`, `detection_time`. Первые строки соответствуют экспериментам с $\beta = 0.1$ (seed 42, 43, 44), где `peak` = 0.000333 — то есть пика фактически нет.

Quarto-документация

— Сгенерированные `.qmd`-файлы рендерились в HTML и открывались в браузере из каталога `markdown/`.

— Файл `sir_migration_effect_literate.html` ([рис. @fig-022]) отображает читаемую документацию с подзаголовком «Формирование матрицы миграции» и блоком кода с подсветкой синтаксиса.

— Файл `sir_run_basic.html` ([рис. @fig-023]) содержит полный словарь параметров в форматированном блоке кода и вывод `Dict{Symbol, Any} with 10 entries`.

— Файл `sir_visualize_dynamics_literate.html` ([рис. @fig-024]) показывает загруженный `DataFrame` 30×13 прямо в документе, с теми же столбцами, что и в ноутбуке.

```
{{< include ../project/markdown/sir_run_basic/sir_run_basic.qmd  
>}}
```

```
{{< include ../project/markdown/sir_scan_beta/sir_scan_beta.qmd  
>}}
```

```
{{< include ../project/markdown/sir_migration_effect/sir_migration_effect.qmd  
>}}
```

Jupyter Notebook `sir_migration_effect_literate.ipynb`

Figure 22: Jupyter Notebook `sir_migration_effect_literate.ipynb`

Jupyter Notebook sir_run_basic.ipynb

Figure 23: Jupyter Notebook sir_run_basic.ipynb

Jupyter Notebook sir_scan_beta_literate.ipynb

Figure 24: Jupyter Notebook sir_scan_beta_literate.ipynb

```
{{< include ../project/markdown/sir_optimize_parameters/sir_optimize_parameters.qmd  
>}}
```

```
{{< include ../project/markdown/sir_visualize_dynamics/sir_visualize_dynamics.qmd  
>}}
```

Использование Git

Работа велась с использованием системы контроля версий Git по модели Gitflow [a@gitflow].

— Репозиторий инициализирован командой `git flow init` с префиксом тегов `v`.

— Разработка велась в ветке `develop`; для каждого логического этапа создавалась `feature`-ветка (например, `feature/sir-model`, `feature/parametric-scan`).

— После завершения этапа ветка сливалась в `develop` командой `git flow feature finish`.

— По завершении лабораторной работы создан релиз:

```
git flow release start 1.0.0  
standard-changelog --first-release  
git add CHANGELOG.md  
git commit -am 'chore(site): add changelog'  
git flow release finish 1.0.0  
git push --all  
git push --tags
```

— Релиз `v1.0.0` опубликован на GitHub с приложением `CHANGELOG.md`:

Jupyter Notebook sir_visualize_dynamics_literate.ipynb: DataFrame
30×13

Figure 25: Jupyter Notebook sir_visualize_dynamics_literate.ipynb:
DataFrame 30×13

Quarto-документ sir_migration_effect_literate.html

Figure 26: Quarto-документ sir_migration_effect_literate.html

Quarto-документ sir_run_basic.html

Figure 27: Quarto-документ sir_run_basic.html

```
gh release create v1.0.0 -F ../release/CHANGELOG.md
```

— Все коммиты оформлены по стандарту Conventional Commits [conventionalcommits]: feat:, fix:, chore:, docs:. Семантическое версионирование [semver] обеспечивает воспроизводимость результатов.

Выводы

В ходе работы реализована и запущена агентная модель SIR для трёх городов по 1000 человек. Базовый эксперимент при $\beta_{\text{und}} = 0.5$ показал, что пик инфицированных достигается примерно на 15-м шаге, после чего численность S падает до нуля, а суммарное население снижается с 3000 примерно до 2600 к концу 100 шагов — около 400 смертей ([рис. @fig-025]).

Параметрическое сканирование по β выявило порог эпидемии между значениями 0.1 и 0.2: при $\beta = 0.1$ пик практически нулевой, при $\beta = 0.2$ — скачкообразно вырастает до 0.33 ([рис. @fig-026]). При $\beta \geq 0.5$ через эпидемию проходит вся популяция, а доля умерших достигает 0.14 при $\beta = 1.0$.

Эксперимент с миграцией показал, что даже небольшая интенсивность 0.1 приводит к тому, что пиковая заболеваемость вырастает с ~ 1000 до ~ 3000 — то есть эпидемия охватывает все три города, а не только третий ([рис. @fig-027]). При дальнейшем увеличении интенсивности значение пика практически не меняется.

Все скрипты успешно преобразованы в литературный стиль [korolkova2025simulation]. Для каждого из них сгенерированы Jupyter Notebook и Quarto-документ, проверена их корректность в браузере. DataFrame с результатами сканирования содержит 30 строк и 13 столбцов ([рис. @fig-021]).

Quarto-документ sir_visualize_dynamics_literate.html

Figure 28: Quarto-документ sir_visualize_dynamics_literate.html

Список литературы